



ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

SGHSS - SISTEMA DE GESTÃO HOSPITALAR

Projeto Multidisciplinar - Desenvolvimento Back-end

Weden Gabriel

RU: 4170826

Londrina - PR

2026

Sumário

1.	Introdução	4
2.	Objetivos do Sistema.....	5
3.	Análise de Requisitos	6
3.1	Requisitos Funcionais	6
3.2	Requisitos Não Funcionais	7
4.	Tecnologias Utilizadas.....	8
5.	Estrutura do Projeto	9
5.1	Diagrama de Caso de Uso	10
5.2	Diagrama Entidade Relacionamento	11
5.3	Diagrama de Classes	12
6.	Funcionalidades Implementadas.....	13
7.	Banco de Dados.....	16
8.	Dockerização	17
9.	Testes Realizados.....	19
9.1	Teste de Autenticação JWT.....	20
9.2	Teste de Cadastro de Usuários.....	21
9.3	Teste de Cadastro de Pacientes	23
9.4	Teste de Cadastro de Médicos	25
9.5	Teste de Cadastro de Consultas	27
10.	Conclusão	32
11.	Referências	33
12.	Link do GitHub.....	33

Lista de Figuras

<i>Figura 1 - Diagrama de Caso de Uso do SGHSS</i>	10
<i>Figura 2 - Diagrama Entidade Relacionamento (DER)</i>	11
<i>Figura 3 - Diagrama de Classes do SGHSS</i>	12
<i>Figura 4 - Execução da aplicação utilizando Docker Compose</i>	18
<i>Figura 5 - Execução da aplicação em container utilizando Docker Desktop</i>	18
<i>Figura 6 - Teste de autenticação JWT no Swagger</i>	20
<i>Figura 7 - Cadastro de usuários via Swagger</i>	21
<i>Figura 8 - Listagem de usuários cadastrados</i>	22
<i>Figura 9 - Cadastro de pacientes via Swagger</i>	23
<i>Figura 10 - Listagem de pacientes cadastrados</i>	24
<i>Figura 11 - Cadastro de médicos via Swagger</i>	25
<i>Figura 12 - Listagem de médicos cadastrados</i>	26
<i>Figura 13 - Cadastro de consultas via Swagger</i>	27
<i>Figura 14 - Listagem de consultas cadastradas</i>	28
<i>Figura 15 - Configuração de segurança com Spring Security</i>	29
<i>Figura 16 - Implementação dos endpoints REST de consultas</i>	30
<i>Figura 17 - Mapeamento da entidade Consulta com JPA/Hibernate</i>	31

Lista de Tabelas

<i>Tabela 1 - Requisitos Funcionais</i>	6
<i>Tabela 2 - Requisitos Não Funcionais</i>	7
<i>Tabela 3 - Tecnologias Utilizadas</i>	8
<i>Tabela 4 - Camadas Utilizadas</i>	9
<i>Tabela 5 - Funcionalidades Implementadas</i>	14
<i>Tabela 6 - Referências</i>	33

1. Introdução

Este trabalho apresenta o desenvolvimento do SGHSS (Sistema de Gestão Hospitalar e de Serviços de Saúde), um sistema criado com foco no gerenciamento de informações hospitalares por meio de uma API REST desenvolvida em Java com Spring Boot.

O projeto foi baseado no estudo de caso proposto pela instituição VidaPlus, que descreve a necessidade de centralizar informações relacionadas a pacientes, profissionais da saúde e consultas médicas em um único sistema.

O foco principal do desenvolvimento foi a parte Back-end da aplicação, trabalhando principalmente na criação da API, autenticação de usuários, estrutura do banco de dados e organização da arquitetura do sistema.

Durante o desenvolvimento foram utilizados recursos bastante comuns no mercado de tecnologia atualmente, como Spring Security, autenticação JWT, Spring Data JPA, Swagger/OpenAPI e Docker.

Além da implementação das funcionalidades, o projeto também buscou aplicar conceitos de Engenharia de Software, organização em camadas, modelagem de banco de dados e documentação técnica da aplicação.

O desenvolvimento do SGHSS contribuiu bastante para colocar em prática conhecimentos vistos ao longo do curso, principalmente relacionados ao desenvolvimento de APIs REST, segurança de aplicações e persistência de dados utilizando Java e Spring Boot.

2. Objetivos do Sistema

O sistema SGHSS tem como objetivo realizar o gerenciamento básico de informações hospitalares através de uma API REST desenvolvida com Java e Spring Boot.

A aplicação foi projetada para permitir o cadastro e gerenciamento de usuários, pacientes, médicos e consultas médicas, utilizando uma arquitetura organizada em camadas e integração com banco de dados relacional.

Além disso, o sistema implementa autenticação utilizando JWT (JSON Web Token), garantindo maior segurança no acesso aos recursos da aplicação.

Outro objetivo do projeto foi aplicar conceitos modernos utilizados no desenvolvimento Back-end profissional, incluindo documentação com Swagger/OpenAPI, persistência de dados com JPA/Hibernate e execução utilizando Docker.

3. Análise de Requisitos

3.1 Requisitos Funcionais

Id	Requisito Funcional	Descrição
RF01	Autenticar usuários	Permitir login utilizando CPF e senha
RF02	Gerenciar perfis	Permitir cadastro e listagem de perfis de acesso
RF03	Gerenciar usuários	Permitir cadastro, listagem, atualização e remoção de usuários
RF04	Gerenciar pacientes	Permitir cadastro e listagem de pacientes vinculados a usuários
RF05	Gerenciar médicos	Permitir cadastro e listagem de médicos vinculados a usuários
RF06	Gerenciar consultas	Permitir cadastro e listagem de consultas médicas
RF07	Persistir dados	Armazenar os dados em banco de dados relacional

Tabela 1 - Requisitos Funcionais

3.2 Requisitos Não Funcionais

Id	Requisito Não Funcional	Descrição
RNF01	Segurança	Utilizar autenticação JWT e criptografia de senha com BCrypt
RNF02	Organização	Utilizar arquitetura em camadas
RNF03	Persistência	Utilizar banco de dados H2 persistente em arquivo
RNF04	Portabilidade	Permitir execução da aplicação com Docker
RNF05	Documentação	Disponibilizar documentação automática com Swagger/OpenAPI
RNF06	Manutenibilidade	Separar responsabilidades entre Controller, Service, Repository, DTO e Entity
RNF07	Versionamento	Utilizar Git e GitHub para controle de versão
RNF08	Usabilidade técnica	Permitir testes da API diretamente pelo Swagger

Tabela 2 - Requisitos Não Funcionais

4. Tecnologias Utilizadas

Durante o desenvolvimento do sistema foram utilizadas tecnologias modernas voltadas ao desenvolvimento Back-end com Java.

Tecnologia	Descrição
Java 21	Linguagem principal utilizada no desenvolvimento da aplicação.
Spring Initializr	Ferramenta utilizada para geração inicial da estrutura do projeto Spring Boot, permitindo configurar dependências e organizar a base da aplicação de forma mais rápida e padronizada.
Spring Web	Responsável pela criação dos endpoints HTTP da API.
Spring Data JPA	Utilizado para persistência de dados e integração com o banco de dados.
Spring Security	Responsável pela configuração de segurança e autenticação da aplicação.
JWT (JSON Web Token)	Tecnologia utilizada para autenticação e geração de tokens de acesso.
H2 Database	Banco de dados utilizado durante o desenvolvimento da aplicação.
Swagger/OpenAPI	Ferramenta utilizada para documentação automática da API REST.
Docker	Utilizado para containerização e execução da aplicação sem necessidade de configuração manual do ambiente Java.
Maven	Ferramenta utilizada para gerenciamento de dependências e build do projeto.

Tabela 3 - Tecnologias Utilizadas

5. Estrutura do Projeto

O sistema foi desenvolvido utilizando arquitetura em camadas, separando responsabilidades para facilitar manutenção, organização e escalabilidade da aplicação.

A estrutura do projeto foi organizada em pacotes específicos para configuração, controle das rotas da API, transferência de dados, entidades do banco de dados, repositórios e regras de negócio.

As principais camadas utilizadas no projeto foram:

Camada	Descrição
Config	Responsável pelas configurações de segurança e autenticação da aplicação, incluindo integração com JWT e Spring Security.
Controller	Responsável pelo recebimento das requisições HTTP da API REST e controle dos endpoints da aplicação.
DTO (Data Transfer Object)	Responsável pela transferência de dados entre cliente e servidor, evitando exposição direta das entidades do banco de dados.
Entity	Representa as tabelas do banco de dados através das entidades JPA/Hibernate.
Repository	Camada responsável pelo acesso e manipulação dos dados no banco utilizando Spring Data JPA.
Service	Responsável pelas regras de negócio da aplicação e processamento das informações.

Tabela 4 - Camadas Utilizadas

A organização em camadas permitiu maior separação de responsabilidades e deixou o projeto mais próximo de padrões utilizados em aplicações profissionais.

5.1 Diagrama de Caso de Uso

O diagrama de caso de uso apresenta as principais interações entre os usuários do sistema e as funcionalidades disponíveis na API. Os atores principais são Administrador, Médico e Paciente, cada um com permissões e responsabilidades relacionadas ao gerenciamento hospitalar.

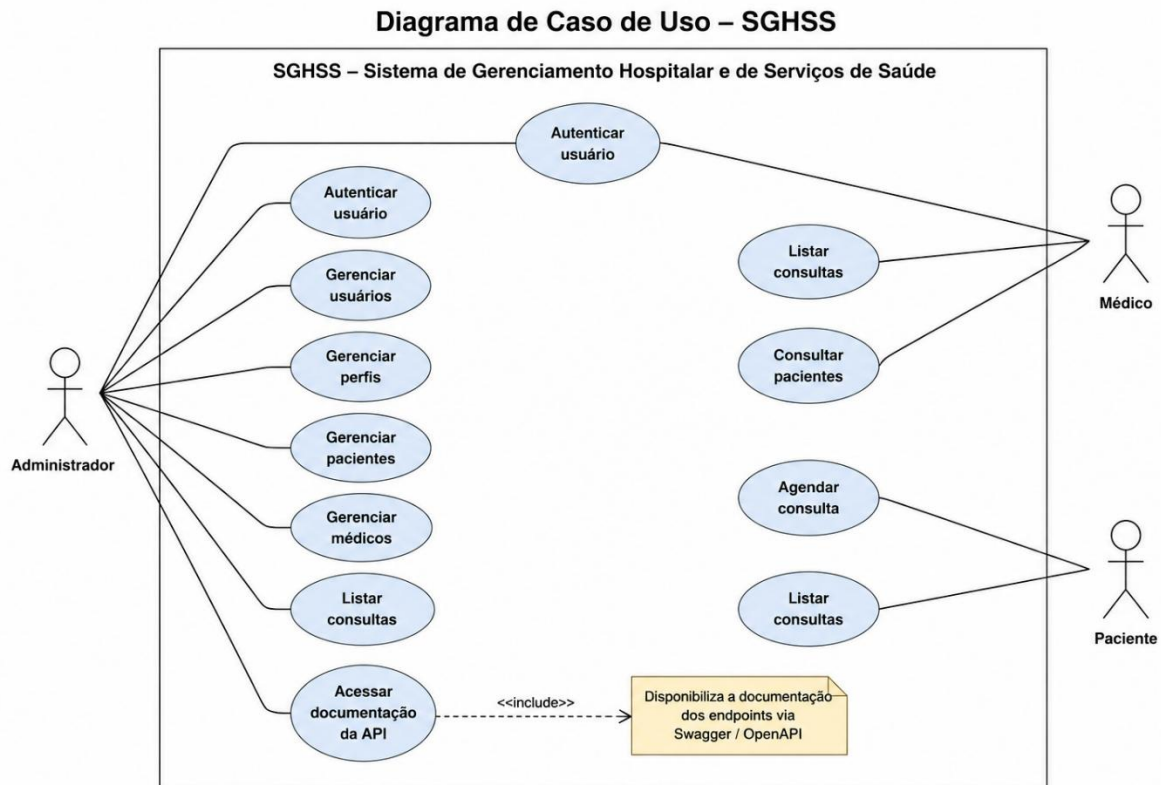


Figura 1 - Diagrama de Caso de Uso do SGHSS

5.2 Diagrama Entidade Relacionamento

O Diagrama Entidade Relacionamento (DER) apresenta a estrutura do banco de dados do SGHSS, demonstrando as entidades principais do sistema, seus atributos e os relacionamentos existentes entre elas.

A modelagem foi desenvolvida com foco na organização das informações hospitalares, permitindo relacionar usuários, pacientes, médicos e consultas médicas de forma estruturada.

Os relacionamentos implementados possibilitam manter integridade e consistência dos dados armazenados pela aplicação.

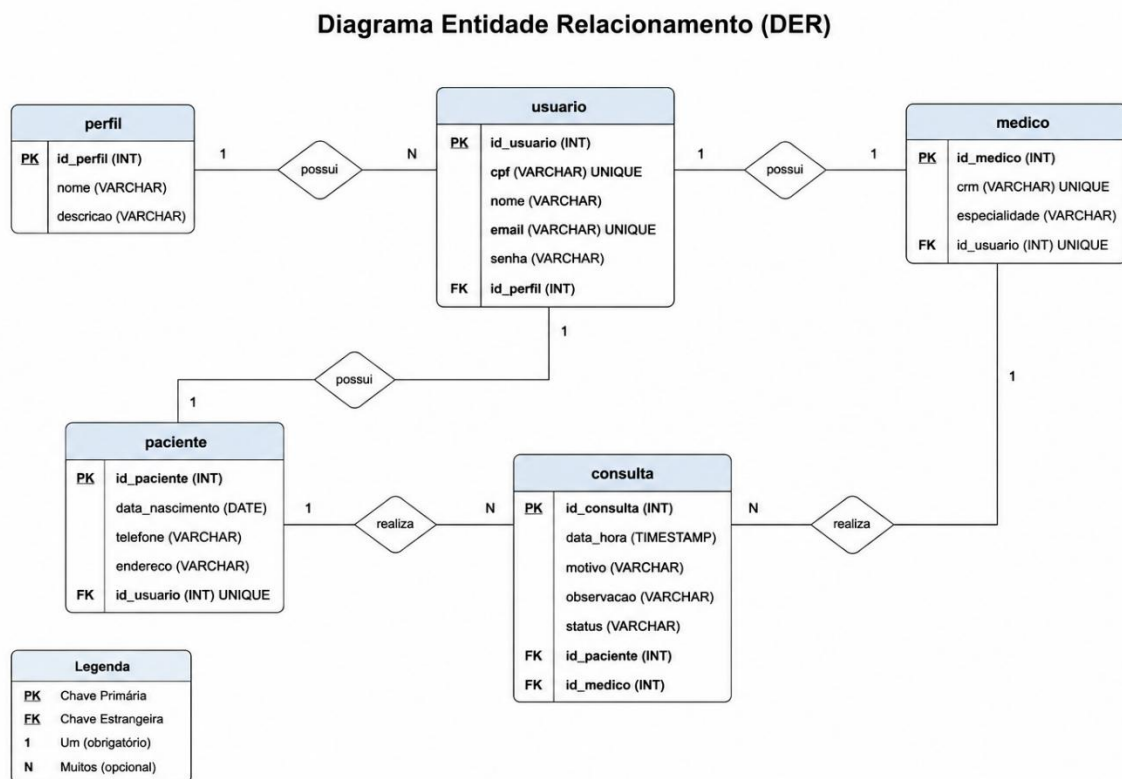


Figura 2 - Diagrama Entidade Relacionamento (DER)

5.3 Diagrama de Classes

O diagrama de classes apresenta a estrutura das principais entidades utilizadas no desenvolvimento do SGHSS, demonstrando atributos, métodos e relacionamentos existentes entre as classes da aplicação.

A modelagem foi baseada na arquitetura Back-end desenvolvida em Java com Spring Boot, permitindo representar de forma visual a organização das entidades responsáveis pelo gerenciamento de usuários, pacientes, médicos e consultas médicas.

Os relacionamentos entre as classes foram definidos considerando a estrutura utilizada na persistência de dados e nas regras de negócio da aplicação.

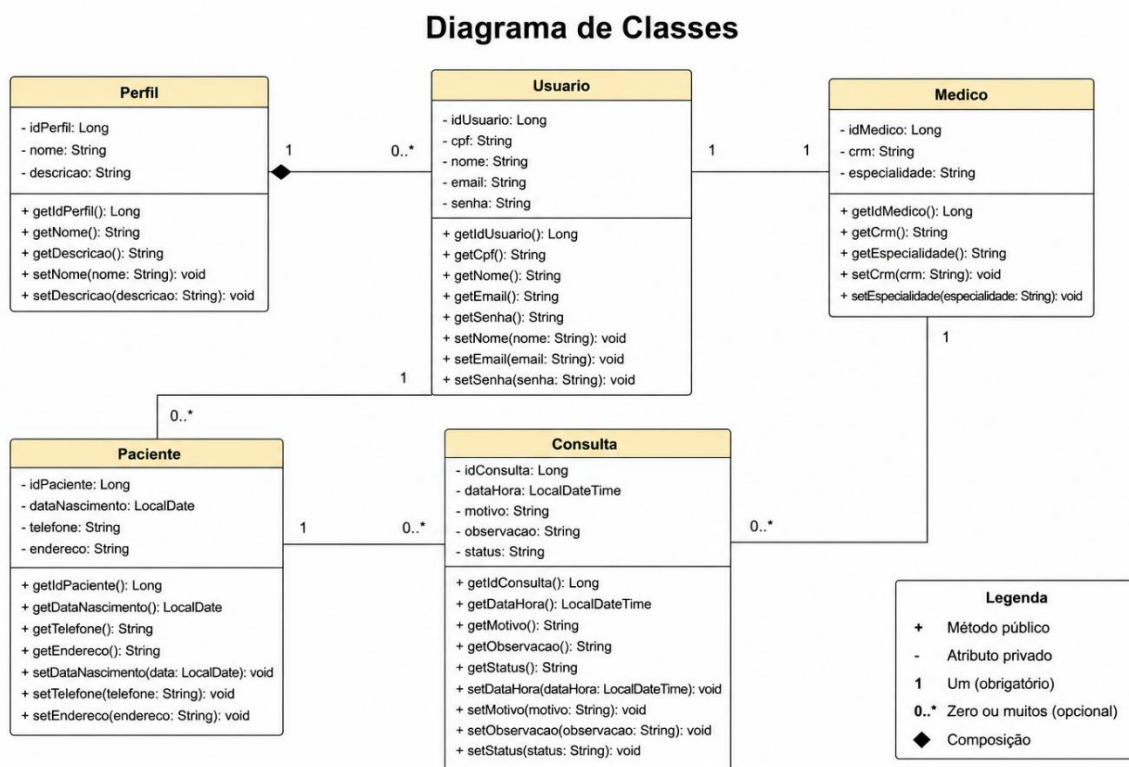


Figura 3 - Diagrama de Classes do SGHSS

6. Funcionalidades Implementadas

Durante o desenvolvimento do sistema foram implementadas funcionalidades essenciais para o gerenciamento hospitalar através de uma API REST.

O sistema permite o cadastro e gerenciamento de usuários, pacientes, médicos e consultas médicas, além de autenticação utilizando JWT.

As principais funcionalidades implementadas foram:

Funcionalidade	Descrição
Autenticação de Usuários	O sistema realiza autenticação através de CPF e senha, retornando um token JWT para acesso aos endpoints protegidos da aplicação.
Cadastro de Perfis	Responsável pelo gerenciamento dos perfis de acesso utilizados no sistema.
Cadastro de Usuários	Responsável pelo gerenciamento de usuários da aplicação, incluindo criação, consulta, atualização e remoção de registros.
Cadastro de Pacientes	Permite registrar pacientes vinculados aos usuários cadastrados no sistema.
Cadastro de Médicos	Responsável pelo gerenciamento de médicos, incluindo especialidade e registro profissional.
Cadastro de Consultas	Permite registrar consultas médicas relacionando pacientes e médicos.
Persistência de Dados	O sistema utiliza banco de dados H2 persistente em arquivo, permitindo manter os dados salvos mesmo após reiniciar a aplicação.
Documentação da API	Foi implementada documentação automática utilizando Swagger/OpenAPI, permitindo visualização e testes dos endpoints diretamente pelo navegador.
Dockerização	O projeto foi configurado para execução através de Docker, permitindo maior portabilidade e facilidade de configuração do ambiente.

Tabela 5 - Funcionalidades Implementadas

A autenticação do sistema foi desenvolvida utilizando JWT (JSON Web Token), permitindo maior segurança no acesso aos endpoints da aplicação.

O processo de autenticação ocorre através do endpoint de login, onde o usuário informa CPF e senha cadastrados no sistema. Após validação das credenciais, a aplicação gera um token JWT que pode ser utilizado para autenticação nas requisições seguintes.

A utilização do JWT permite que a aplicação mantenha autenticação de forma segura sem necessidade de armazenamento de sessão no servidor.

Durante o desenvolvimento foram utilizados recursos do Spring Security para configuração da segurança da aplicação e BCrypt para criptografia de senhas.

O endpoint utilizado para autenticação foi:

POST /auth/login

Exemplo de requisição:

<pre>{ "cpf": "99999999999", "senha": "123" }</pre>

Exemplo de resposta:

<pre>{ "token": "jwt-token" }</pre>

A implementação da autenticação JWT aproximou o projeto de padrões modernos utilizados no desenvolvimento de APIs profissionais.

7. Banco de Dados

O sistema utiliza o banco de dados H2 para armazenamento das informações da aplicação.

O H2 foi escolhido por ser um banco leve, simples de configurar e muito utilizado em projetos acadêmicos e ambientes de desenvolvimento.

A persistência dos dados foi implementada utilizando Spring Data JPA juntamente com Hibernate, permitindo o mapeamento das entidades Java para tabelas do banco de dados.

As principais entidades implementadas no sistema foram:

- Usuário
- Perfil
- Paciente
- Médico
- Consulta
- Prontuário
- Log de Auditoria

Os relacionamentos entre as entidades foram configurados utilizando anotações JPA, permitindo associação entre usuários, pacientes, médicos e consultas médicas.

O banco foi configurado para persistência em arquivo, permitindo manter os dados armazenados mesmo após reiniciar a aplicação.

Também foi utilizada a ferramenta H2 Console para visualização e gerenciamento do banco durante o desenvolvimento do projeto.

Configuração utilizada:

JDBC URL: jdbc:h2:file:./data/sghss
User: sa
Password: (vazio)

A utilização do Spring Data JPA facilitou significativamente as operações de persistência e manipulação dos dados da aplicação.

8. Dockerização

Durante o desenvolvimento do projeto foi implementada a dockerização da aplicação utilizando Docker e Docker Compose.

A utilização do Docker permitiu criar um ambiente padronizado para execução do sistema, eliminando a necessidade de instalação manual de dependências como Java e Maven na máquina do usuário.

Foram criados os arquivos Dockerfile, docker-compose.yml e .dockerignore para configuração e gerenciamento dos containers da aplicação.

Com a dockerização, o sistema pode ser executado utilizando apenas o comando:

```
docker compose up --build
```

Após a primeira execução, a aplicação pode ser iniciada novamente utilizando:

```
docker compose up
```

Para encerramento dos containers foi utilizado:

```
docker compose down
```

A utilização do Docker trouxe maior portabilidade ao projeto, facilitando testes, distribuição e execução da aplicação em diferentes ambientes

A aplicação foi containerizada utilizando Docker e Docker Compose, permitindo a criação automatizada do ambiente Back-end da API.

A Figura 4 apresenta o processo de build e inicialização da aplicação utilizando Docker Compose através do terminal.

A Figura 5 apresenta a execução da aplicação em container utilizando Docker Desktop, demonstrando os logs de inicialização do Spring Boot e da conexão com o banco de dados.

```
#12 resolving provenance for metadata file
#12 DONE 0.0s
[+] up 3/3
✓ Image sghss-java-sghss-api Built 46.7s
✓ Network sghss-java_default Created 0.0s
✓ Container sghss-api Created 0.2s
Attaching to sghss-api
sghss-api |
sghss-api |
sghss-api |      _/_--_-'-----(-)--- _/\_/_\
sghss-api | ( ) \_--_|'_'|'|'_|'_/\_/_\_\
sghss-api | \|_--_| |_) | | | | | | C | | ) ) )
sghss-api | ' |____|_|_| | | | | |__/_/_/_/
sghss-api | =====|_|=====|___/_/_/_/
sghss-api |
sghss-api | :: Spring Boot ::                (v3.5.14)
sghss-api | 2026-05-08T01:59:14.294Z INFO 1 --- [backend] [          main] com.sghss.backend.BackendApplication :
sghss-api | Starting BackendApplication v0.0.1-SNAPSHOT using Java 21.0.10 with PID 1 (/app/target/backend-0.0.1-SNAPSHOT.jar start
ed by root in /app)
sghss-api | 2026-05-08T01:59:14.296Z INFO 1 --- [backend] [          main] com.sghss.backend.BackendApplication :
sghss-api | No active profile set, falling back to 1 default profile: "default"
sghss-api | 2026-05-08T01:59:15.101Z INFO 1 --- [backend] [          main] .s.d.r.c.RepositoryConfigurationDelegate :
sghss-api | Bootstrapping Spring Data JPA repositories in DEFAULT mode.
sghss-api | 2026-05-08T01:59:15.160Z INFO 1 --- [backend] [          main] .s.d.r.c.RepositoryConfigurationDelegate :
sghss-api | Finished Spring Data repository scanning in 51 ms. Found 7 JPA repository interfaces.
sghss-api | 2026-05-08T01:59:15.710Z INFO 1 --- [backend] [          main] o.s.b.w.embedded.tomcat.TomcatWebServer :
sghss-api | Tomcat initialized with port 8080 (http)
sghss-api | 2026-05-08T01:59:15.724Z INFO 1 --- [backend] [          main] o.apache.catalina.core.StandardService :
```

Figura 4 - Execução da aplicação utilizando Docker Compose

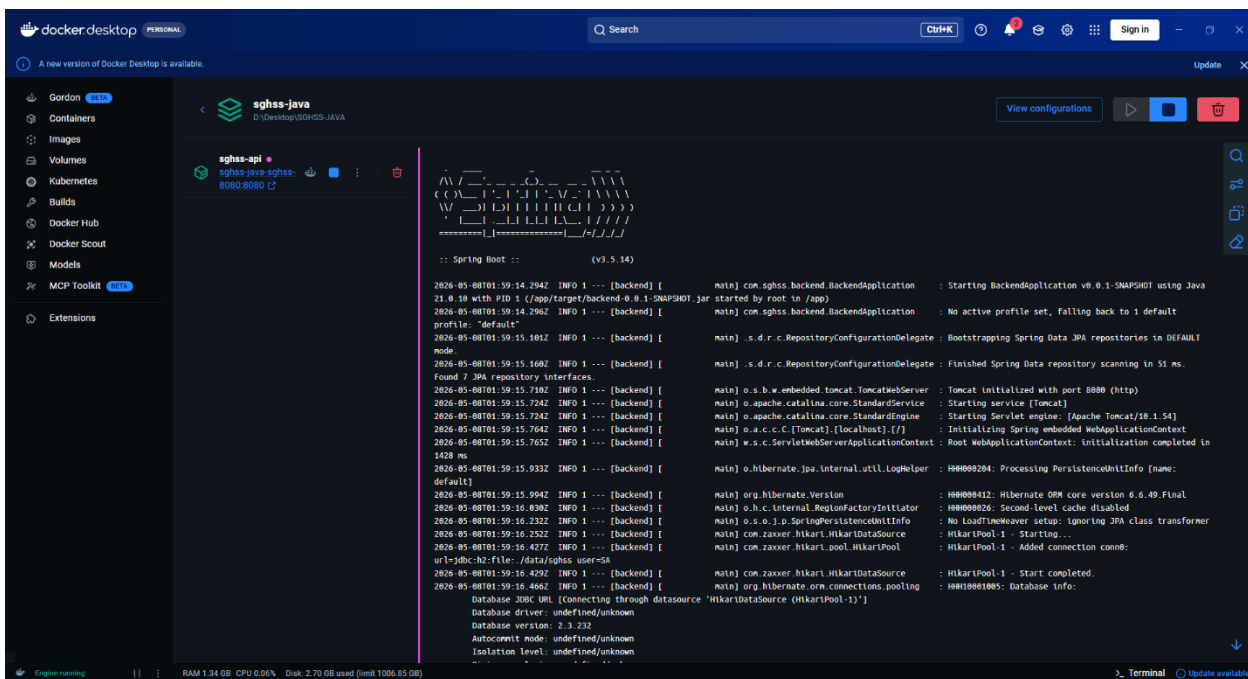


Figura 5 - Execução da aplicação em container utilizando Docker Desktop

9. Testes Realizados

Durante o desenvolvimento do sistema foram realizados testes utilizando Swagger/OpenAPI para validação dos endpoints da API REST.

Os testes permitiram validar o funcionamento das operações de cadastro, listagem, autenticação e relacionamento entre as entidades do sistema.

Os principais testes realizados foram:

9.1 Teste de Autenticação JWT

Foi realizado teste do endpoint de autenticação utilizando CPF e senha válidos, retornando corretamente o token JWT utilizado para acesso aos endpoints protegidos da aplicação.

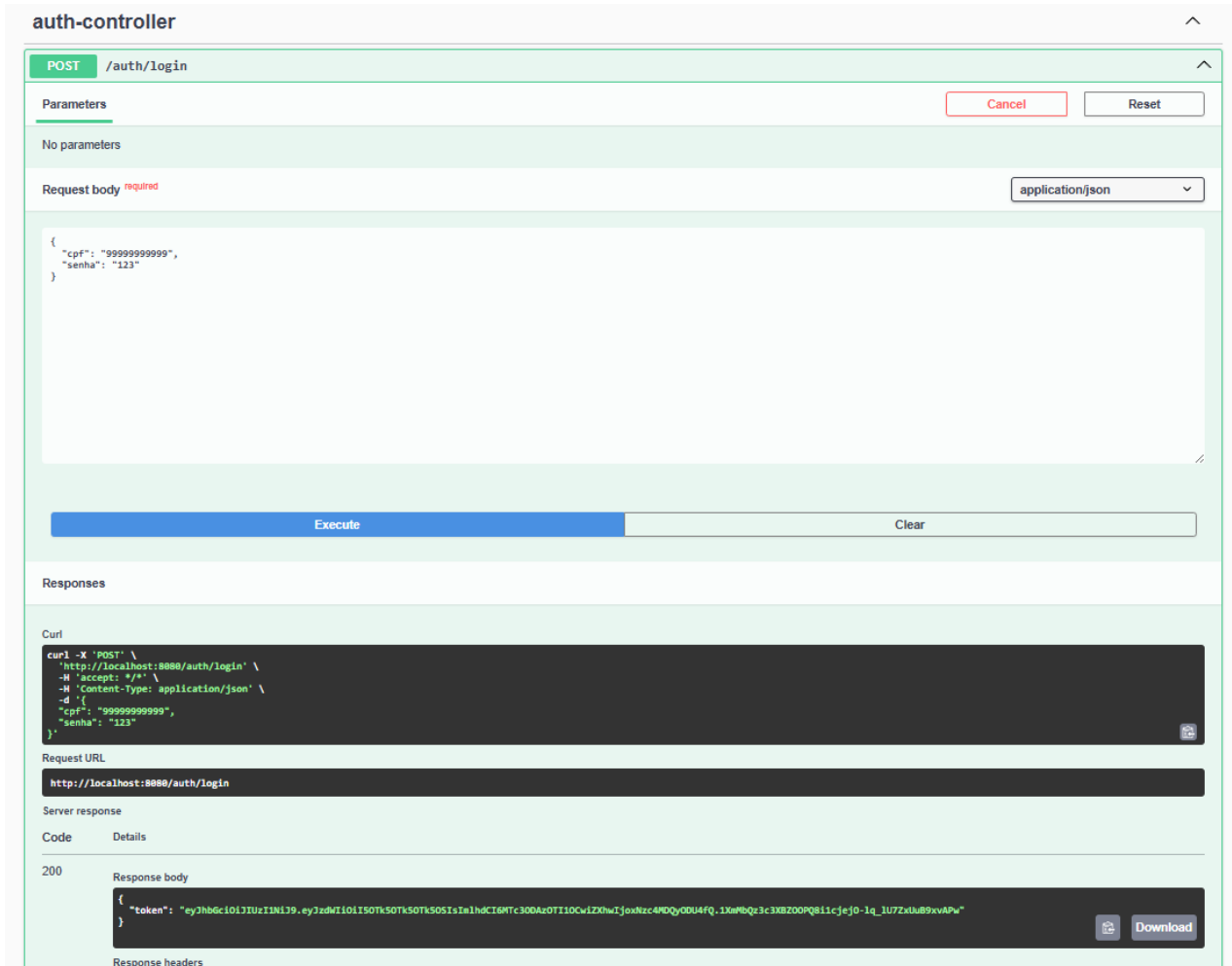
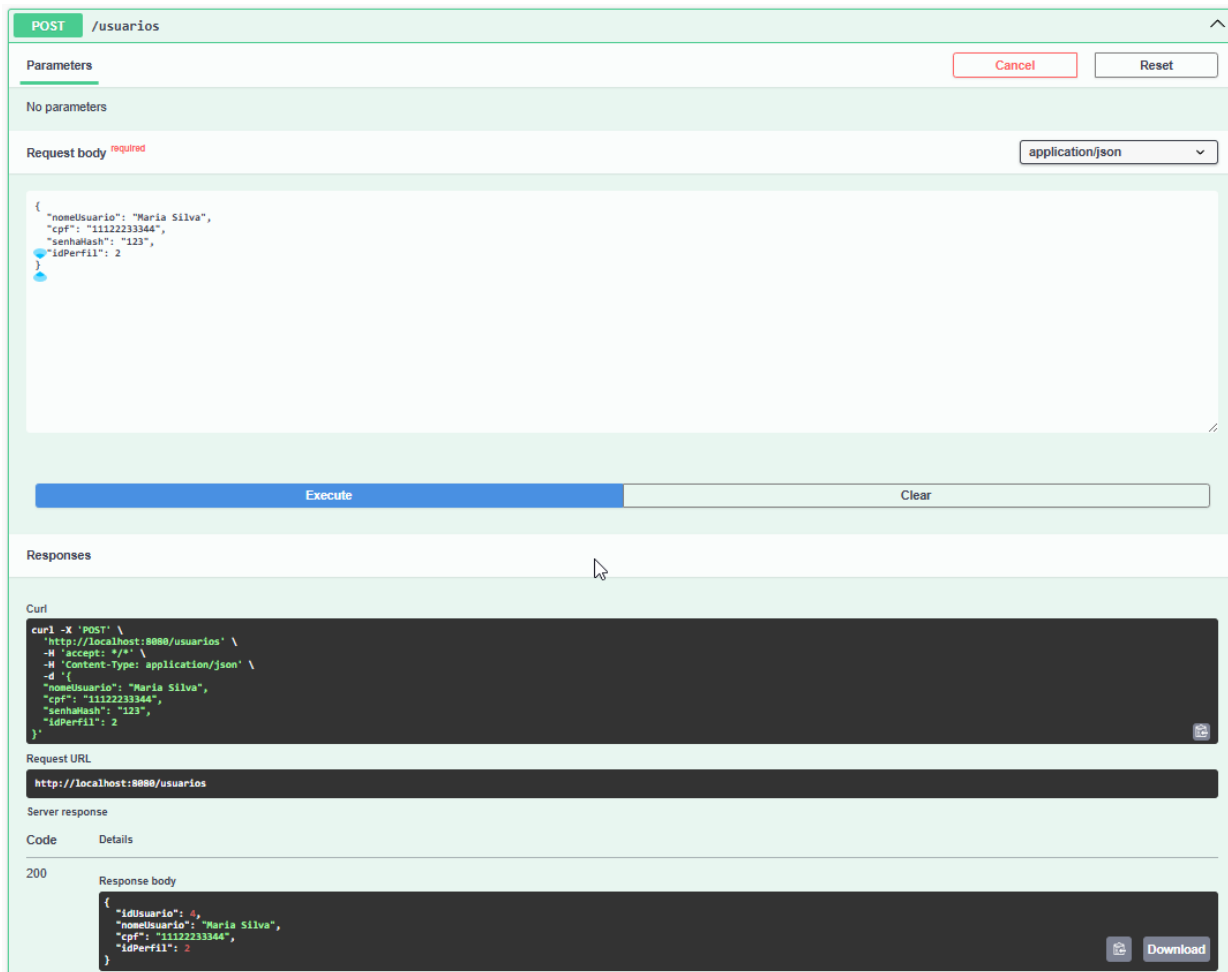


Figura 6 - Teste de autenticação JWT no Swagger

9.2 Teste de Cadastro de Usuários

Foi realizado teste do endpoint de cadastro de usuários utilizando requisição POST no Swagger, validando o armazenamento correto das informações no sistema.



The screenshot displays the Swagger UI interface for the POST /usuarios endpoint. The 'Parameters' section is empty. The 'Request body' is set to 'application/json' and contains the following JSON payload:

```
{
  "nomeUsuario": "Maria Silva",
  "cpf": "11122233344",
  "senhaHash": "123",
  "idPerfil": 2
}
```

The 'Execute' button is highlighted in blue. Below the request, the 'Responses' section shows a successful response with a status code of 200. The response body is:

```
{
  "idUsuario": 4,
  "nomeUsuario": "Maria Silva",
  "cpf": "11122233344",
  "idPerfil": 2
}
```

The interface also includes a 'Curl' section with the following command:

```
curl -X 'POST' \
  'http://localhost:8080/usuarios' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nomeUsuario": "Maria Silva",
    "cpf": "11122233344",
    "senhaHash": "123",
    "idPerfil": 2
  }'
```

The 'Request URL' is 'http://localhost:8080/usuarios'. The 'Server response' section shows the status code '200' and the response body.

Figura 7 - Cadastro de usuários via Swagger

Foi realizado também teste de listagem dos usuários cadastrados utilizando requisição GET, confirmando o retorno correto das informações armazenadas no banco de dados.

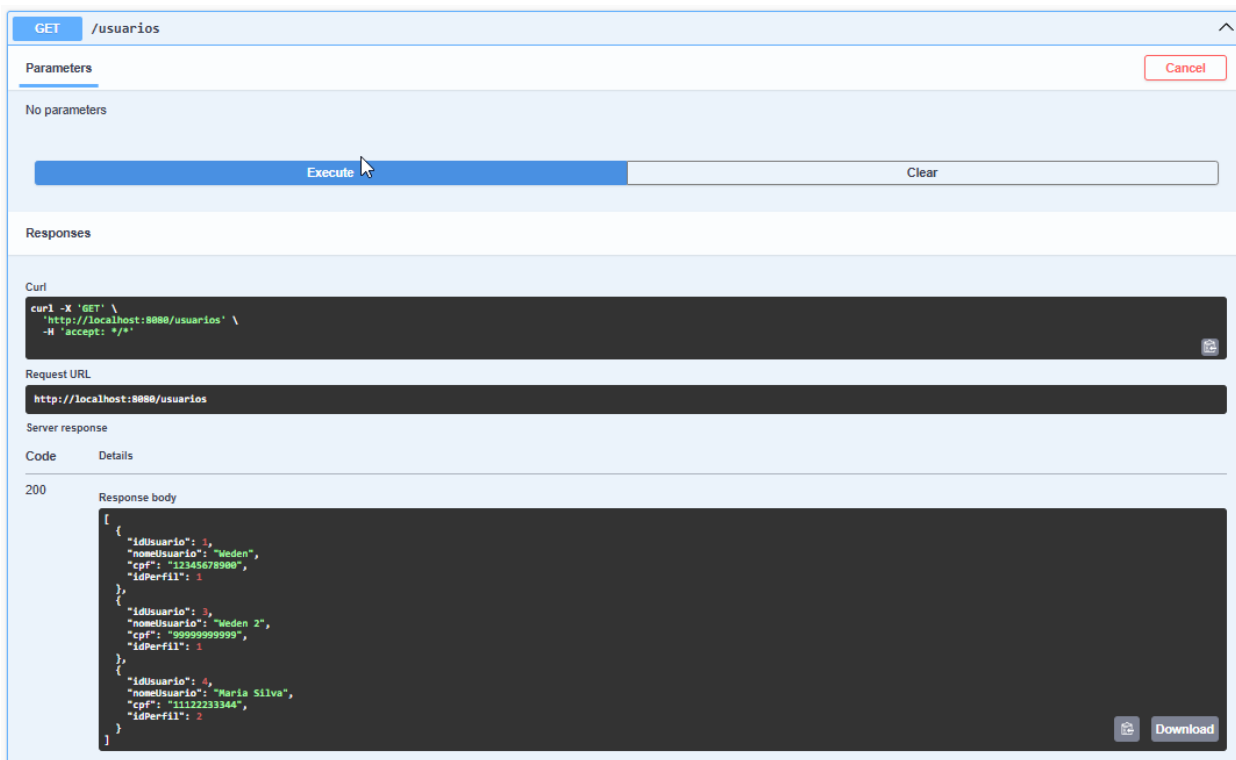
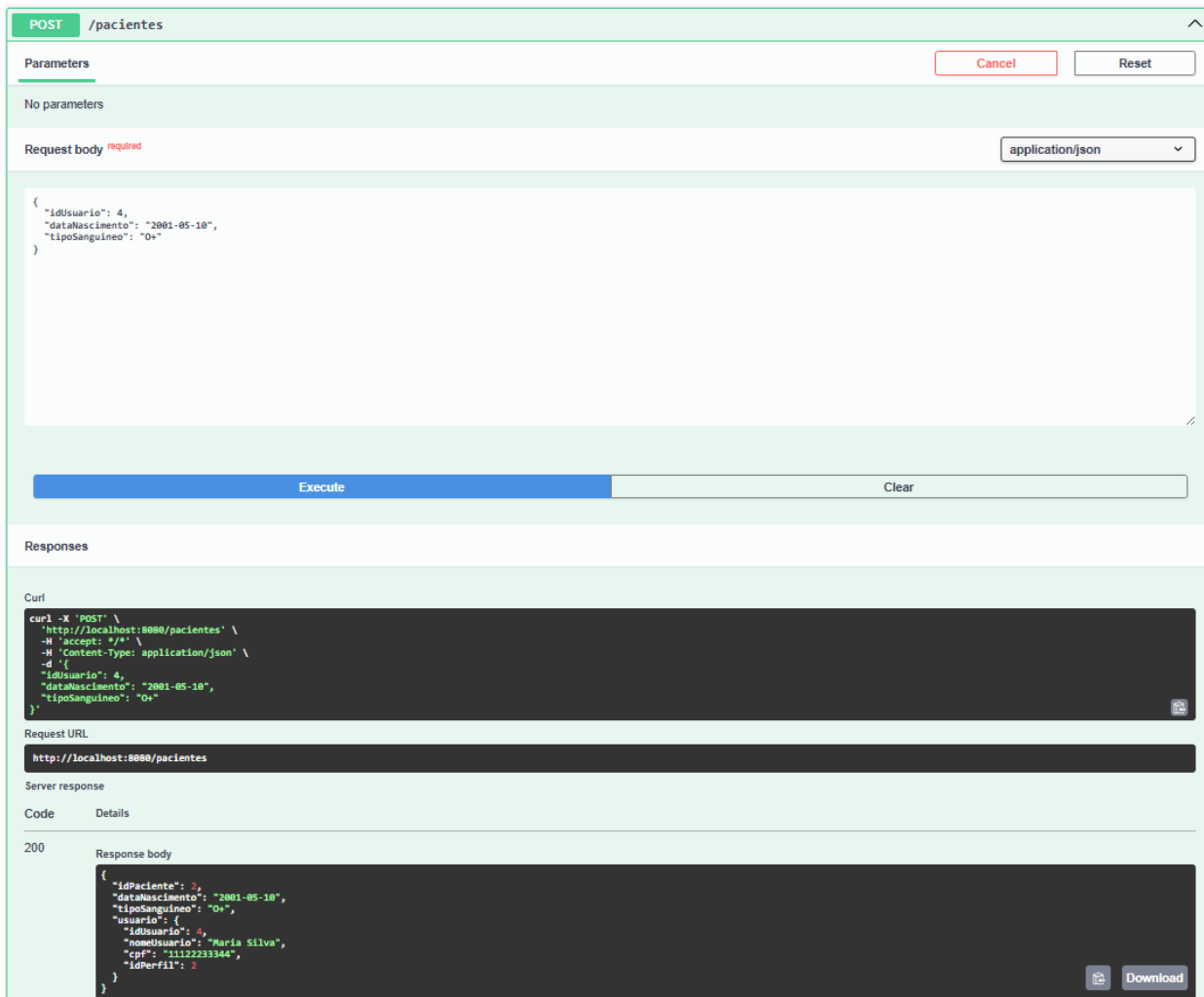


Figura 8 - Listagem de usuários cadastrados

9.3 Teste de Cadastro de Pacientes

Foi realizado teste do endpoint de cadastro de pacientes utilizando requisição POST no Swagger, validando o relacionamento entre paciente e usuário previamente cadastrado no sistema.



POST /pacientes

Parameters

No parameters

Request body *required* application/json

```
{
  "idUsuario": 4,
  "dataNascimento": "2001-05-10",
  "tipoSanguineo": "O+"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/pacientes' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "idUsuario": 4,
    "dataNascimento": "2001-05-10",
    "tipoSanguineo": "O+"
  }'
```

Request URL

http://localhost:8080/pacientes

Server response

Code Details

200

Response body

```
{
  "idPaciente": 2,
  "dataNascimento": "2001-05-10",
  "tipoSanguineo": "O+",
  "usuario": {
    "idUsuario": 4,
    "nomeUsuario": "Maria Silva",
    "cpf": "11122233344",
    "idPerfil": 2
  }
}
```

Download

Figura 9 - Cadastro de pacientes via Swagger

Foi realizado também teste de listagem dos pacientes cadastrados utilizando requisição GET, confirmando o retorno correto das informações armazenadas no banco de dados.

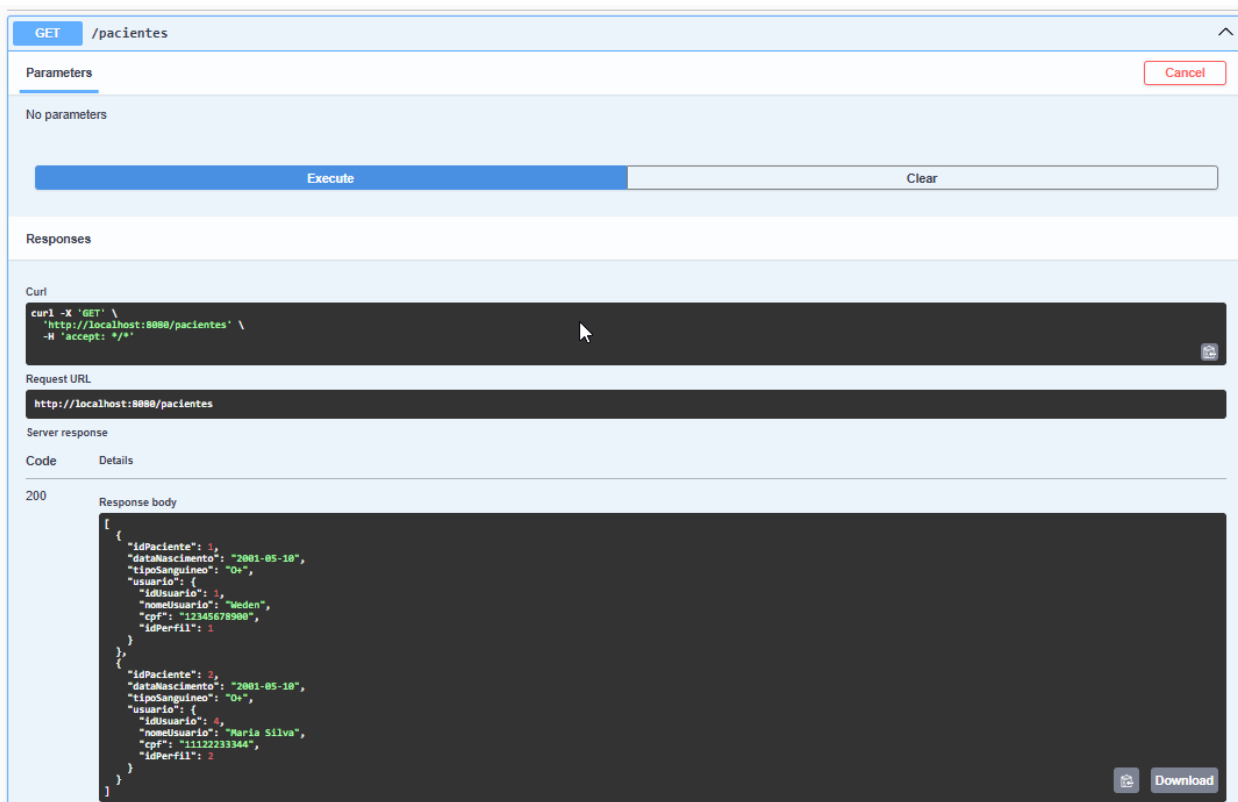
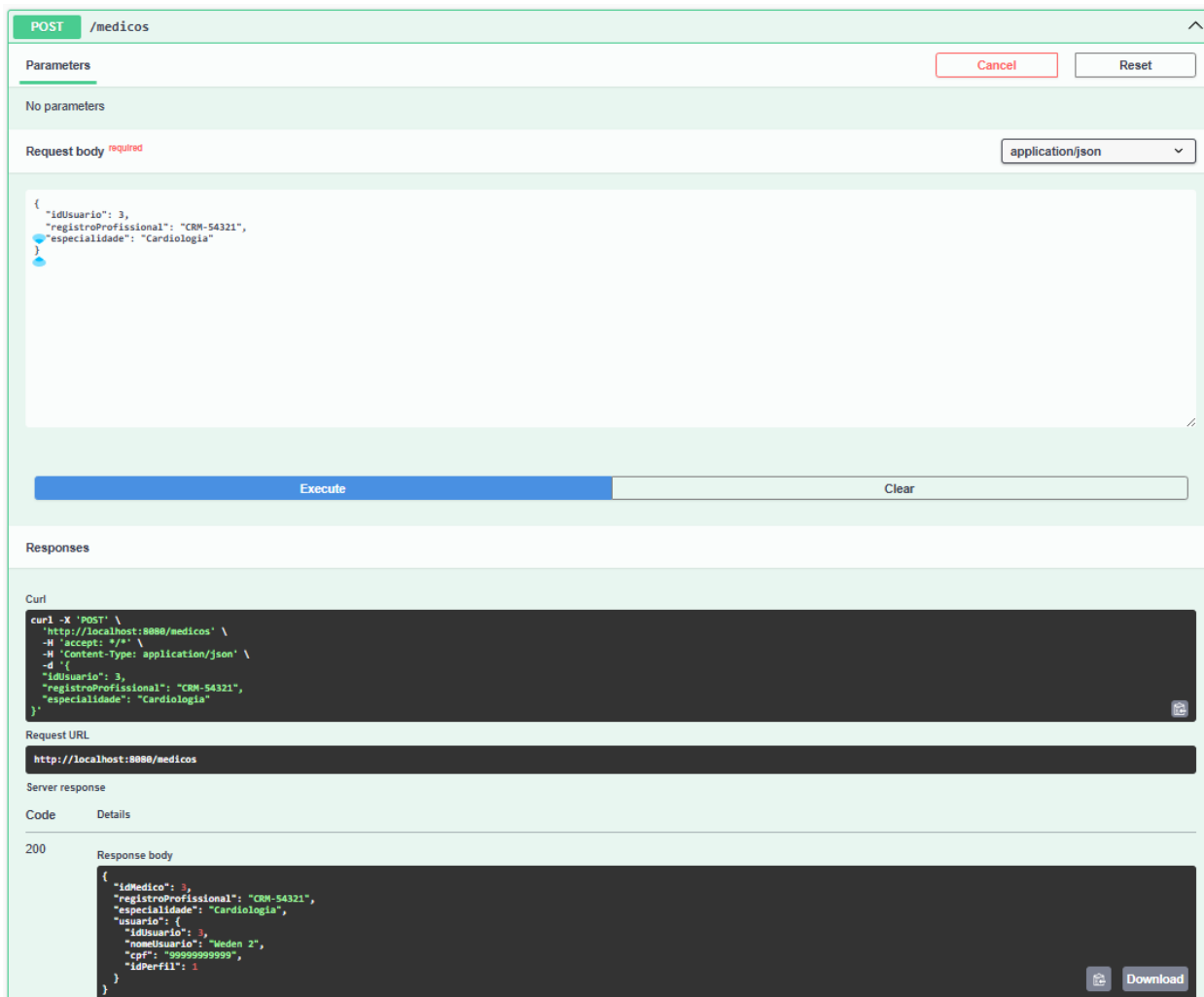


Figura 10 - Listagem de pacientes cadastrados

9.4 Teste de Cadastro de Médicos

Foi realizado teste do endpoint de cadastro de médicos utilizando requisição POST no Swagger, validando o armazenamento das informações profissionais e especialidades médicas no sistema.



POST /medicos

Parameters

No parameters

Request body *required* application/json

```
{
  "idUsuario": 3,
  "registroProfissional": "CRM-54321",
  "especialidade": "Cardiologia"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/medicos' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "idUsuario": 3,
    "registroProfissional": "CRM-54321",
    "especialidade": "Cardiologia"
  }'
```

Request URL

http://localhost:8080/medicos

Server response

Code Details

200

Response body

```
{
  "idMedico": 3,
  "registroProfissional": "CRM-54321",
  "especialidade": "Cardiologia",
  "usuario": {
    "idUsuario": 3,
    "nomeUsuario": "Meden 2",
    "cpf": "99999999999",
    "idPerfil": 1
  }
}
```

Download

Figura 11 - Cadastro de médicos via Swagger

Também foi realizado teste de listagem dos médicos cadastrados através de requisição GET, verificando o retorno correto dos dados registrados.

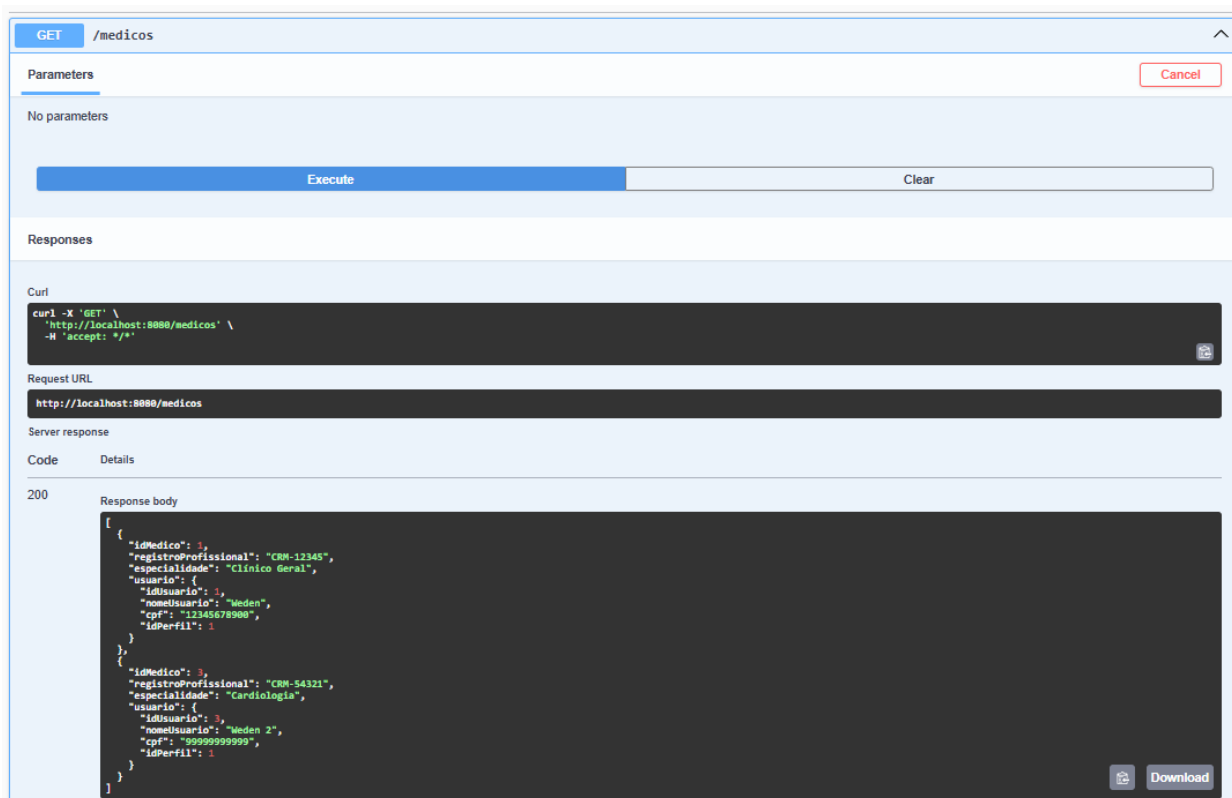
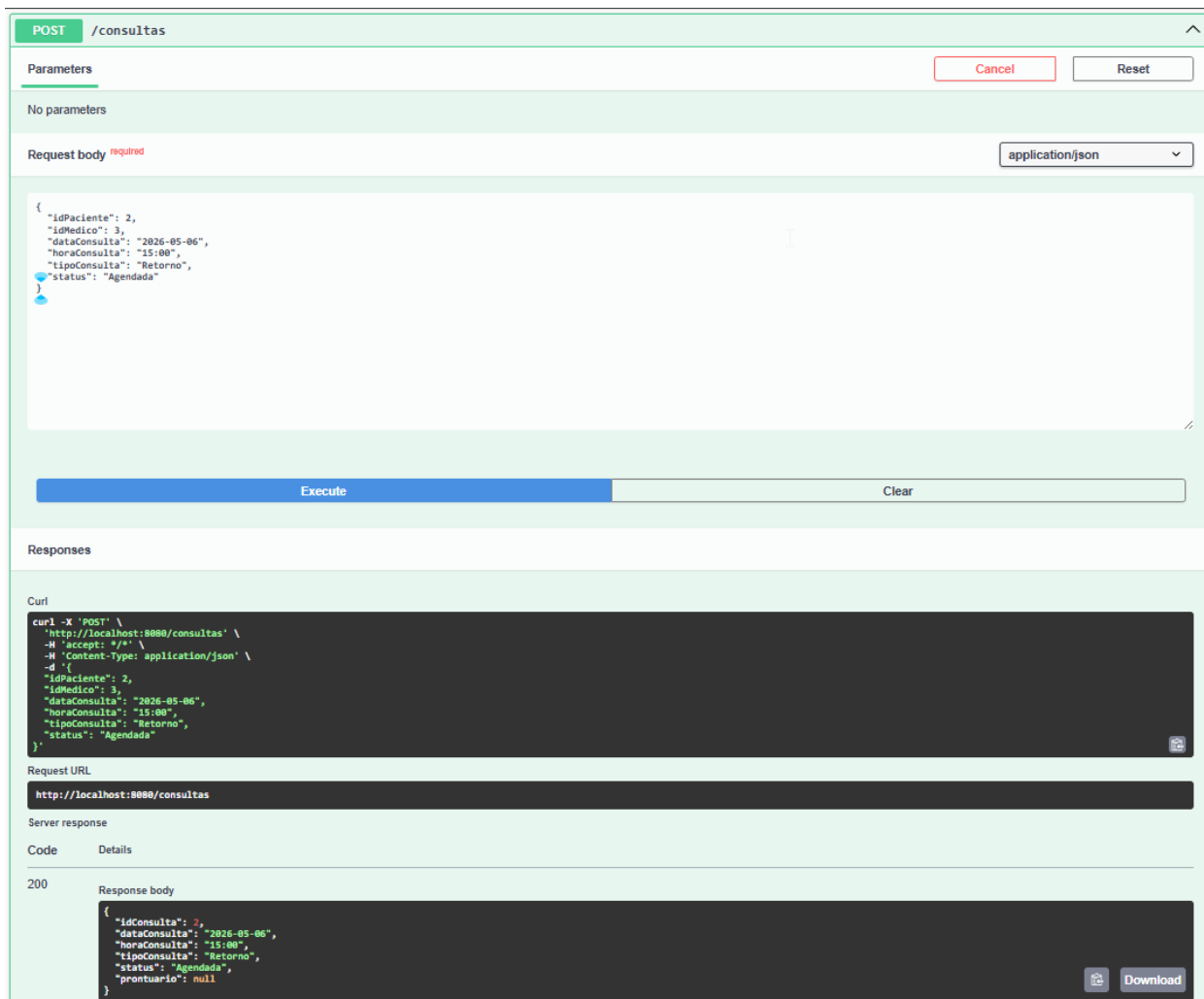


Figura 12 - Listagem de médicos cadastrados

9.5 Teste de Cadastro de Consultas

Foi realizado teste do endpoint de cadastro de consultas utilizando requisição POST no Swagger, validando o relacionamento entre pacientes e médicos cadastrados anteriormente.



POST /consultas

Parameters

No parameters

Request body *required* application/json

```
{
  "idPaciente": 2,
  "idMedico": 3,
  "dataConsulta": "2026-05-06",
  "horaConsulta": "15:00",
  "tipoConsulta": "Retorno",
  "status": "Agendada"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/consultas' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "idPaciente": 2,
    "idMedico": 3,
    "dataConsulta": "2026-05-06",
    "horaConsulta": "15:00",
    "tipoConsulta": "Retorno",
    "status": "Agendada"
  }'
```

Request URL

http://localhost:8080/consultas

Server response

Code	Details
200	Response body <pre>{ "idConsulta": 2, "dataConsulta": "2026-05-06", "horaConsulta": "15:00", "tipoConsulta": "Retorno", "status": "Agendada", "prontuario": null }</pre> Download

Figura 13 - Cadastro de consultas via Swagger

Também foi realizado teste de listagem das consultas cadastradas através de requisição GET, verificando o retorno correto das informações registradas no sistema.

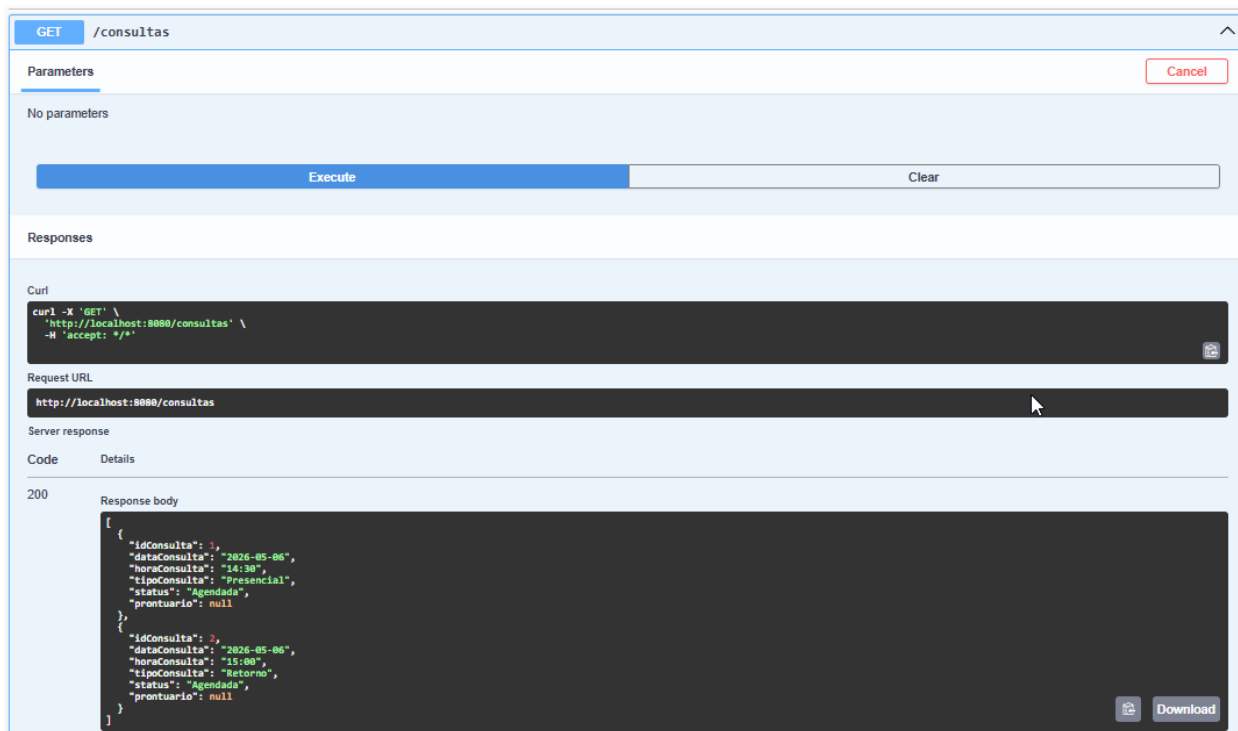


Figura 14 - Listagem de consultas cadastradas

Os testes realizados demonstraram o correto funcionamento das funcionalidades implementadas durante o desenvolvimento do projeto.

Além dos testes realizados pelo Swagger, foram registrados exemplos da implementação do código-fonte, demonstrando a configuração de segurança, criação de endpoints REST e mapeamento das entidades utilizadas na persistência dos dados.

SecurityConfig.java

```

1  package com.sghss.backend.config;
2
3  import org.springframework.context.annotation.Bean;
4  import org.springframework.context.annotation.Configuration;
5  import org.springframework.security.config.annotation.web.builders.HttpSecurity;
6  import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
7  import org.springframework.security.crypto.password.PasswordEncoder;
8  import org.springframework.security.web.SecurityFilterChain;
9
10 @Configuration
11 public class SecurityConfig {
12
13     @Bean
14     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
15         http
16             .csrf(csrf → csrf.disable())
17             .headers(headers → headers
18                 .frameOptions(frame → frame.disable())
19             )
20             .authorizeHttpRequests(auth → auth
21                 .anyRequest().permitAll()
22             );
23
24         return http.build();
25     }
26
27     @Bean
28     public PasswordEncoder passwordEncoder() {
29         return new BCryptPasswordEncoder();
30     }
31 }

```

Figura 15 - Configuração de segurança com Spring Security

ConsultaController.java

```

1  package com.sghss.backend.controller;
2
3  import java.util.List;
4
5  import org.springframework.web.bind.annotation.DeleteMapping;
6  import org.springframework.web.bind.annotation.GetMapping;
7  import org.springframework.web.bind.annotation.PathVariable;
8  import org.springframework.web.bind.annotation.PostMapping;
9  import org.springframework.web.bind.annotation.RequestBody;
10 import org.springframework.web.bind.annotation.RequestMapping;
11 import org.springframework.web.bind.annotation.RestController;
12
13 import com.sghss.backend.dto.ConsultaCreateDTO;
14 import com.sghss.backend.dto.ConsultaDTO;
15 import com.sghss.backend.service.ConsultaService;
16
17 import jakarta.validation.Valid;
18
19 @RestController
20 @RequestMapping("/consultas")
21 public class ConsultaController {
22
23     private final ConsultaService consultaService;
24
25     public ConsultaController(ConsultaService consultaService) {
26         this.consultaService = consultaService;
27     }
28
29     @PostMapping
30     public ConsultaDTO criarConsulta(@RequestBody @Valid ConsultaCreateDTO dto) {
31         return consultaService.criarConsulta(dto);
32     }
33
34     @GetMapping
35     public List<ConsultaDTO> listarConsultas() {
36         return consultaService.listarConsultas();
37     }
38
39     @GetMapping("/{id}")
40     public ConsultaDTO buscarPorId(@PathVariable Integer id) {
41         return consultaService.buscarPorId(id);
42     }
43
44     @DeleteMapping("/{id}")
45     public void deletarConsulta(@PathVariable Integer id) {
46         consultaService.deletarConsulta(id);
47     }
48 }

```

Figura 16 - Implementação dos endpoints REST de consultas

Consulta.java

```

1 package com.sghss.backend.entity;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.GeneratedValue;
6 import jakarta.persistence.GenerationType;
7 import jakarta.persistence.Id;
8 import jakarta.persistence.JoinColumn;
9 import jakarta.persistence.ManyToOne;
10 import jakarta.persistence.Table;
11
12 @Entity
13 @Table(name = "consulta")
14 public class Consulta {
15
16     @Id
17     @GeneratedValue(strategy = GenerationType.IDENTITY)
18     private Integer idConsulta;
19
20     @ManyToOne
21     @JoinColumn(name = "id_paciente")
22     private Paciente paciente;
23
24     @ManyToOne
25     @JoinColumn(name = "id_medico")
26     private Medico medico;
27
28     @Column(name = "data_consulta")
29     private String dataConsulta;
30
31     @Column(name = "hora_consulta")
32     private String horaConsulta;
33
34     @Column(name = "tipo_consulta")
35     private String tipoConsulta;
36
37     private String status;
38
39     // GETTERS E SETTERS
40
41     public Integer getIdConsulta() {
42         return idConsulta;
43     }
44
45     public void setIdConsulta(Integer idConsulta) {
46         this.idConsulta = idConsulta;
47     }
48
49     public Paciente getPaciente() {
50         return paciente;
51     }
52
53     public void setPaciente(Paciente paciente) {
54         this.paciente = paciente;
55     }
56
57     public Medico getMedico() {
58         return medico;
59     }
60
61     public void setMedico(Medico medico) {
62         this.medico = medico;
63     }
64
65     public String getDataConsulta() {
66         return dataConsulta;
67     }
68
69     public void setDataConsulta(String dataConsulta) {
70         this.dataConsulta = dataConsulta;
71     }
72
73     public String getHoraConsulta() {
74         return horaConsulta;
75     }
76
77     public void setHoraConsulta(String horaConsulta) {
78         this.horaConsulta = horaConsulta;
79     }
80
81     public String getTipoConsulta() {
82         return tipoConsulta;
83     }
84
85     public void setTipoConsulta(String tipoConsulta) {
86         this.tipoConsulta = tipoConsulta;
87     }
88
89     public String getStatus() {
90         return status;
91     }
92
93     public void setStatus(String status) {
94         this.status = status;
95     }
96 }

```

Figura 17 - Mapeamento da entidade Consulta com JPA/Hibernate

10. Conclusão

O desenvolvimento do sistema SGHSS permitiu aplicar na prática diversos conceitos importantes relacionados ao desenvolvimento Back-end com Java e Spring Boot.

Durante o projeto foram utilizados conceitos de arquitetura em camadas, criação de APIs REST, persistência de dados com JPA/Hibernate, autenticação utilizando JWT e documentação automática com Swagger/OpenAPI.

Além disso, a utilização do Docker proporcionou maior portabilidade para execução da aplicação, aproximando o projeto de práticas utilizadas no mercado profissional.

O sistema desenvolvido atingiu os objetivos propostos, permitindo o gerenciamento de usuários, pacientes, médicos e consultas médicas através de uma API funcional e organizada.

O projeto também contribuiu significativamente para evolução prática em tecnologias Back-end, integração com banco de dados, segurança de autenticação e organização estrutural de aplicações Java.

A experiência adquirida durante o desenvolvimento do SGHSS fortaleceu conhecimentos importantes para continuidade dos estudos e evolução profissional na área de desenvolvimento de software.

Durante o desenvolvimento do projeto também foram encontrados desafios relacionados à organização da arquitetura Back-end, autenticação JWT, integração entre entidades e containerização da aplicação utilizando Docker.

Esses desafios contribuíram significativamente para aprofundar conhecimentos em desenvolvimento Java, APIs REST e boas práticas utilizadas no mercado de tecnologia.

11. Referências

Descrição	Referência
SPRING. Spring Boot Documentation	https://spring.io/projects/spring-boot
JWT.IO. JSON Web Tokens	https://jwt.io/
DOCKER. Docker Documentation	https://docs.docker.com/
H2 DATABASE. H2 Database Engine	https://www.h2database.com/html/main.html
SWAGGER. OpenAPI Specification	https://swagger.io/specification/

Tabela 6 - Referências

12. Link do GitHub

<https://github.com/wedengabriel/sistema-gestao-hospitalar-sghss>